CSE 574 Final Report

Goal-Oriented Action Planning with Reinforcement Learning Adversarial Agents in a Discrete 2D Environment

Rowan Holder
School of Computing and Augmented Intelligence (SCAI)
Arizona State University (ASU)
Tempe, Arizona, USA
rwholde1@asu.edu

Abstract

This paper showcases a pursuit-evasion simulation set on a 16-by-16 discrete grid environment. In this cat-and-mouse scenario, A Goal-Oriented Action Planner (GOAP) acts as a cat, and a Reinforcement Learning agent utilizing tabular Q-Learning acts as a mouse. The GOAP has several different actions and constraints that can be toggled, including a pounce, a rest cycle, and energy to manage. It's main movement planning is based on A*. The RL agent learns to evade the GOAP inside a local observation window which can be scaled up or down. It's reward function is designed specifically to encourage kiting: a key strategy in video games to stay just out of reach of enemies while maintaining control of their movement and (in most games) dealing damage to the enemy. The agent can be trained in a static environment or generalized using a random environment. This paper explains the system design and implementation, discusses the RL agent's reward function, training, and tuning for kiting behavior, and explore the limitations of tabular Q-learning at the environment's scale as well as directions for future work.

1 Introduction

This project explores a fundamental problem in artificial intelligence: autonomous agents planning in an adversarial environment. This exploration has applications throughout robotics, within multi-agent systems, and Game AI. In particular, pursuit-evasion problems, or cat-and-mouse games, provide a simple environment to evaluate the behaviors of autonomous agents in an adversarial environment. The simulation built and described in this paper tasks a Goal-Oriented Action Planner (GOAP) with catching a Reinforcement Learning Agent as it tries to learn to avoid it.

Goal-Oriented Action Planners are used in modern game development as an alternative to finite state machines and behavior trees [4]. GOAP agents dynamically plan actions that will satisfy their goal states which in turn produce legible behaviors that designers can understand while being flexible enough for the same designers to mold them for their particular task. The planner was first introduce in the video game F.E.A.R. in 2005, and it has been used for game AI ever since.

The weakness of the planner, and their benefit within games, is that they are deterministic and, by extension, easily exploited by players. This exploitation is generally referred to as "kiting", where a player will control an enemy's movement by being just near enough to know it will follow but not so near that it can damage or catch the player. This interaction inspired the prospect that a learning agent could also exploit a GOAP. As such, this project implemented a simulation to test variations of the pursuit-evasion problem by allowing users to enable and disable GOAP actions, affect the environment both agents would maneuver through, and change training hyperparameters of the RL agent.

This report will be structured in the following way. Section 2 will explore the background on Goal-Oriented Action Planning, Q-Learning, and previous work with pursuit-evasion problems. Next, the system design will be discussed in section 3, which explains the environment, how each agent works within the simulation, the use of A* within the GOAP for optimal navigation, and decisions related to which values the user should be able to interact with. After this overview, specific implementation decisions will be explained in section 4 related to training the RL agent and backend visualization decisions. Results and Analysis will compare how agents respond to different environments or when they have access to different actions or constraints. Next, the Discussion in section 6 to cover the limitations in scalability of the simulation. Finally, the paper will be concluded in section 7.

2 Background and Related Work

2.1 Goal-Oriented Action Planning (GOAP)

Jeff Orkin introduced Goal-Oriented Action Planning (GOAP) as a planning paradigm with the video game F.E.A.R (2005) as a scalable alternative to hand-authored Finite State Machines (FSMs) in his talk at the Game Developers Conference in 2006 [4]. In his paper, Orkin explains that the GOAP consists of three components. First, they have world state, which is a predicate set that describes the agent and the environment. Second, the agent has a set of actions it can take. In GOAP, actions are defined by their preconditions and their effects.

Last, the GOAP has a Goal state that includes the target predicate assignments that the planner seeks to achieve. When planning, the GOAP utilizes A* in order to find the lowest cost action sequence that will transition the agent from the current world state to its defined goal state, performing a forward-search over the actions to find this sequence. Having a cost associated with an action is a direct departure from STRIPS planning, and it is what allow A* to be used to find the lowest cost action sequence that will take the agent to the goal state. Another notable difference is the lack of add and delete lists for GOAP. This is replaced by having the state show all relevant predicates that can be changed by any actions an agent can take and modifying the predicates based on the action accordingly. Preconditions and effects in Goal-Oriented Action Planning are procedural unlike in STRIPS. Procedural effects are made temporal by connecting the planner to a finite state machine and sequentially activating actions from the plan rather than making an instantaneous change to the world state. Procedural preconditions help by allowing the planner to get access to information it needs without overloading it with information.

Goal-Oriented Action Planning was a big departure from Finite State Machines and Behaviors Trees previously used in video games. Finite State Machines denote specific state transitions for their AI, while behavior trees provide a hierarchical and modular logic structure, allowing for AI behavior to be more complex and reactive. In contrast, by using a dynamic planner in order to have AI generate action sequences to reach goals, GOAP creates emergent action sequences by deriving valid transitions from action preconditions. These behaviors are even more complex while still being understandable and tunable by designers.

2.2 Reinforcement Learning – Q-Learning

Q-Learning is a Model-Free Reinforcement Learning algorithm, meaning the agent learns purely from their experience taking actions in the world [2]. It utilizes off-policy Temporal Difference methods in order to evaluate and change its policy. The algorithm includes several key components:

- $\mathcal{S}$: The State Space
- $\mathcal{A}$: The Action Space
- s: The current State
- a: The Action taken
- $\mathcal{R}$(s, a, s'): The Reward Function
- Q(s, a): The Q-Function

The State Space $\mathcal{S}$ denotes all the agent's available states. Likewise, the Action Space $\mathcal{A}$ is the set of the agent's available actions. The Reward Function $\mathcal{R}$(s, a, s') returns a reward value based on the current state, the action taken, and the state after the action is taken. The Q-Function Q(s, a) defines the expected rewards for taking a particular action in a given state. The outputs of the Q-Function are called Q-values, with the best value for each state-action-state pair being stored in a Q-table, which constitutes the agent's memory. The Q-table is updated using the Temporal Difference algorithm.

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[\mathcal{R} + \gamma \max_{a' \in \mathcal{A}} Q(s',a') - Q(s,a) \right] \quad (1)$$

This algorithm also includes a few hyperparameters. $\alpha \in (0, 1]$ is learning rate. This value determines how likely the agent is to overwrite their policy. $\gamma \in [0, 1]$ is the discount factor, which helps set the agent's focus on long-term rewards. The final hyperparameter is $\epsilon \in [0, 1]$, which is the exploration factor. This value decides how likely the agent is to take a new action over the policy action. This value is initialized to be a high value, usually between 0.9 and 1, which allows the agent to explore their environment and develop new strategies by learning new actions and writing the rewards to the Q-table. The Q-Learning agent is trained over a pre-decided number of episodes, and each episode can be given a maximum number of steps to ensure the agent doesn't run forever. Over the course of training, the exploration factor is annealed such that the agent will being choosing actions based on their knowledge stored in the Q-table to maximize its rewards in a process called exploitation.

2.3 Pursuit-Evasion Problems

Pursuit-Evasion are a classic planning problem type that involve a pursuing agent that attempts to minimize the time it takes to capture an evader while that evader agent aims to maximize its survival time [3]. With extensive research through multiple formulations of these problems, the best strategies for pursuit and evasion are well known. One such evasion strategy is known as "kiting", where the evader remains just outside the pursuer's range while still controlling their movement by making them follow. This is utilized against game AI to great effect; players will kite enemies while they defeat them so they don't take damage while defeating the enemy. Kiting is most effective against agents with mechanics that limit their ability to chase the evader, including cooldowns or resource depletion. These mechanics can easily be exploited by the evader as in many cases they can close distance against the pursuer while they are limited and safely retreat before they have recovered [1].

3 System Design

3.1 Environment

The simulated environment is a discrete 16-by-16 grid world. This specified the pursuit-evasion problem as discrete by extension. Each cell is either assigned as free or having a wall, with walls being impassable cells. The walls are generated stochastically based on a density parameter $\rho \in [0, 0.5]$ that is set by the user and is initialized to 0.25. When the GOAP has the Energy limitation active, a food cell is placed in the center of the map ($x, y \in [7, 9]$) and is guaranteed to

be passable. The GOAP agent is initialized at $(1, 1)$ and the RL agent begins at $(14, 14)$. Around each spawn point is an exclusion zone where cells are always initialized as free. The environment is episodic for both agents. For the GOAP, the environment is fully observable while the RL agent observes a local window based on a window size parameter $\phi \times \phi, 7 \leq \phi \leq 13, \phi \equiv 1(mod2)$ that can be changed by the user. The environment can either be fixed between episodes, where the a map is generated at the start of the RL agent's training and used throughout and in testing, or randomized between episodes, where a new map is generated for every training episode and every test run. This parameter can be set by the user when initializing the environment.

3.2 GOAP Agent (Cat)

The GOAP's world state is comprised by a vector containing boolean and numeric predicates. Below is a list of the agent's defined actions.

- `scan_env`: If the environment has not been scanned, the environment is scanned and env_scanned is set to true.
- `locate_goal`: If the environment has been scanned, find the goal within the environment. When it is found, set goal_located to true.
- `compute_path`: If the goal is located, find a path from the GOAP's position to the goal position, then set path_computed to true.
- `pounce`: If the path has been computed, the cooldown counter is 0, and the agent has at least 5 energy, move 3 spaces in one direction, set the cooldown counter to 4, and use 5 energy.
- `rest`: If the agent has moved 5 times contiguously, the agent is rests for two cycles by not moving.
- `execute_move`: If the path has been computed and the cooldown counter is 0, the agent moves one space in a direction and uses one energy.

The GOAP has 3 optional modules that can be toggled by the user. The first is Pounce. The agent evaluates a 3-cell forward jump along the calculated A* path will land on the goal space (i.e. where the RL agent is) or skips enough path steps to make the 4-cycle cooldown cost worthwhile. The optimal final position for a pounce action first evaluates if there is a jump that lands on the goal before choosing the highest net-saving jump. The second optional module is Rest. This requires the agent to rest for two cycles every 5 moves for which time it can not move. The last optional module is Energy. This module give the agent an energy bar of 80 points to maintain. Each step, the GOAP uses 1 energy, and for each pounce it uses 5 energy. When the agent runs out of energy, its goal switches to the food cell. Once the agent steps on the food cell, its energy bar is refilled.

3.3 RL Agent (Mouse)

3.3.1 State Space. The prey observes a local state vector composed of:

- Relative position to the GOAP: Clamped to an $n \times n$ window (default $n = 9$), encoded as two integers $\Delta x, \Delta y \in [-\lfloor n/2 \rfloor, \lfloor n/2 \rfloor]$.
- GOAP pounce cooldown: Cooldown for if the GOAP agent has used the pounced action recently. Bucketed to $[0, 4]$.
- GOAP rest count: If the GOAP is resting, and for how long. Bucketed to $[0, 2]$.
- GOAP energy. How much energy the GOAP has. Bucketed into 5 bands of 20.
- Mobility count: How many spaces the RL agent can move to from its current position. Clamped to $[0, 4]$.

The total number of discrete states is $n^2 \times 5 \times 3 \times 5 \times 5 = 3{,}375$ for the default 9×9 window.

3.3.2 Action Space. The RL agent can move in any of the four cardinal directions. The agent only considers actions that would move it to a passable cell. Furthermore, at each step, the agent is required to move; it may not be stationary.

3.3.3 Reward Function. The agent's reward function is comprised of a terminal signal and two per-step shaping components:

$$\mathcal{R}(s, a, s') = \begin{cases} -200 & \text{if caught} \\ +300 & \text{if survived} \\ \mathcal{R}_{\text{z}}(d) + \mathcal{R}_{\text{mobility}}(m) & \text{otherwise} \end{cases} \tag{2}$$

Where $d = \|p_{\text{RL}} - p_{\text{GOAP}}\|_1$ denotes the Manhattan distance between the two agents and m is the RL agent's Mobility count. The zone reward attempts to inspire the agent to utilize kiting behavior.

$$\mathcal{R}_{\text{z}}(d) = \begin{cases} +3 & \text{if } d_{\min} \leq d \leq d_{\max} \\ -(d_{\min} - d) \times 4 & \text{if } d < d_{\min} \\ \max(0,\ 2 - (d - d_{\max}) \times 0.3) & \text{if } d > d_{\max} \end{cases} \tag{3}$$

With this function, the agent attempts to stay in the distance band $[d_{\min}, d_{\max}] = [3, 6]$. The agent stays a safe distance away from the GOAP while still maintaining control of its movement; too close and the agent is penalized, while staying too far away gives the agent diminishing rewards. In turn, the mobility penalty discourages the agent from moving into corners or closed-off areas.

$$\mathcal{R}_{\text{mobility}}(m) = \begin{cases} -3.0 & \text{if } m \leq 1 \\ -1.0 & \text{if } m = 2 \\ 0 & \text{if } m \geq 3 \end{cases} \tag{4}$$

Kiting enemies is easier when the space around the agent is open. By penalizing the agent for positioning itself near

walls, it makes the agent more likely to move to open areas and for the kiting behavior desired to emerge.

3.4 Spatial Navigation – A*

A* is used compute the path taken by the GOAP agent, whether that be towards the RL agent or moving to get food and refill its energy. The heuristic used by the algorithm is Manhattan distance: $h(n) = |n_x - g_x| + |n_y - g_y|$. Wall are excluded from the neighbor expansion, and A* is re-invoked for the GOAP agent each step, resulting in a fully reactive path plan.

4 Implementation

4.1 Q-Table

The Q-table is stored as a hash map that associates a state key string with a four-element array of action values, one per cardinal direction, all initialized to zero. State keys are constructed by concatenating the bucketed state dimensions with underscore delimiters, which includes relative position, pounce cooldown, rest count, energy band, and mobility count. This structure allows lookups and updates to run in constant average time regardless of how many entries the table grows to over training. When the RL agent visits a state that has no entry in the table, the entry is created on first access and initialized to all zeros. This lazy initialization means memory use grows only as states are actually visited rather than pre-allocating the full theoretical state space.

4.2 Training Loop

Training runs for a configurable number of episodes between 500 and 5,000. Within each episode, both agents are reset to their starting positions and the GOAP's energy, cooldown, and sprint counter are all returned to their initial values. At every step the RL agent selects an action, the GOAP responds with its planned step, rewards are computed, and the Q-table is updated via Equation 1. An episode terminates when the RL agent is caught or when the step count reaches the 250-move limit.

Training episodes for the agent are processed in synchronous batches of 10 within an asynchronous outer loop. Between batches, a zero-millisecond timeout is inserted to yield control back to the browser event loop, keeping the interface responsive and allowing the reward chart to update incrementally during training rather than only at the end. This approach trades a small amount of wall-clock training time for a live view of convergence.

The exploration factor ϵ is annealed linearly from 1.0 down to a floor of 0.05 over the full episode budget. Early in training the agent explores freely, writing new Q-values across a wide range of states. As ϵ degrades as it is annealed, the agent increasingly follows its current best policy, which allows the reward curve to reflect genuine policy improvement rather than continued random exploration.

4.3 Environment Snapshots

Each rendered playback frame carries a deep copy of the grid and the food cell coordinates that were active at the start of that episode. This is necessary because the global grid is mutated in place at the beginning of every episode when environment randomization is enabled. Without per-frame snapshots, the canvas during playback would always reflect whichever map happened to be loaded last rather than the one the agents were actually navigating. By embedding the snapshot directly in the frame object at the time it is produced, the visualizer can render the correct maze for every step of playback regardless of how many subsequent episodes have been run.

4.4 GOAP Planner

The GOAP planner runs a forward priority-queue search over the action graph at the start of each episode to produce the high-level plan sequence: scan, locate, compute path, and then select between pounce and execute move based on the modules that are active. The cost of each action is fixed, so the planner always returns the same sequence for a given module configuration; its role is to confirm the valid action ordering rather than to react dynamically each step.

Spatial navigation, by contrast, is fully reactive. A* is re-invoked every step with the RL agent's current position as the goal, so the GOAP's path plan is always optimal for where the RL agent is right now rather than where it was when the episode began. When the energy module is active and the GOAP is exhausted, the goal passed to A* switches from the RL agent's position to the food cell. The path is re-calculated again the following step, which means the GOAP immediately re-routes back toward the RL agent the moment its energy is replenished.

4.5 User-Configurable Parameters

Several parameters that meaningfully affect agent behavior are exposed to the user through the interface. The wall density ρ and the RL observation window size ϕ both require a full environment reset before they take effect because they change the structure of the grid and the dimensionality of the Q-table respectively. Changing either mid-training would invalidate all existing Q-values. The number of training episodes and the environment randomization mode, on the other hand, take effect immediately on the next training run because they only affect how many episodes are processed and which map is loaded at the start of each one. The three GOAP modules—pounce, rest, and energy—are also applied immediately, though they also alter the RL agent's effective state space. Toggling modules between training runs without resetting the Q-table will therefore produce a mismatch between the states recorded in the table and the states the agents will visit going forward.

Table 1. Agent response to different scenarios (listed below), including which agent was dominant and how many training episodes there were in which the RL agent survived.

Scenario*	Winner	Training Survival
A	RL	0-12/500
B	RL	0-5/500
C	GOAP (Sometimes RL)	0-2/500
D	Tie (RL Favored)	45-78/1000
E	Tie (GOAP Favored)	0-3/1000

*see 5.1

5 Results and Analysis

The resulting system is a modular adversarial environment where a GOAP agent attempt to catch an RL agent as it is trained to avoid its pursuer. This simulation can be modified and tuned to visualize how the two agent's will respond to different settings and, by extension, how effective each are in different situations. Table 1 reports on a few different scenarios and how the agent's performed in each.

5.1 Scenarios

- A: No modules, static environment
- B: No modules, dynamic environment
- C: Pounce+Rest+Energy, static environment
- D: Rest+Energy, dynamic environment
- E: Pounce+Rest+Energy, dynamic environment

In all scenarios, Wall Density ρ was 0.25 and Window Size was 9×9.

5.2 Analysis

5.2.1 Scenario A. Observing Scenario A reveal how easily exploitable the GOAP is by the RL agent. Because the cat is so deterministic in its movement, the mouse can easily exploit it and generate movement loops that allow it to survive. Even, so, it takes a fair amount of time for the RL agent to learn how to exploit the GOAP, with only a maximum of about a 2% survival rate during training.

5.2.2 Scenario B. Scenario B reveals generalizable strategies the RL agent tends to learn to survive the GOAP agent. In general, the mouse either kites the cat to an open corner and oscillates to loop the cat's movement and survive, or the mouse kites the cat such that the mouse ends up 4 spaces in front of cat, retaining the maximum reward spacing while staying safe. It then traverses in a circle around the edge of the map, understanding that walls are not able to spawn in these locations.

5.2.3 Scenario C. In this scenario, the GOAP gains the upper hand as it is forced to break up its deterministic movement with the Rest and Energy modules. This makes the cat less predictible for the mouse, so the RL agent has a more difficult time developing a policy. The RL agent rarely survives at all during the training stage, but the behavior of kiting can still sometimes be observed.

5.2.4 Scenario D. While the RL agent has its highest success finding a policy during training that generalizes, there are still times when the GOAP gets the better of it. This seems to stem from the nondeterministic nature of the rest module, which forces the mouse to kite the cat across the map or develop more advanced oscillations to retain an action set that can be looped.

5.2.5 Scenario E. The RL agent is much less likely to survive during training when the GOAP agent is able to use its pounce action. However, when shown using a its greedy policy, the mouse is sometimes able to survive the cat. There are just more situations where the RL agent ends up 3 spaces away from the GOAP agent and is caught.

5.2.6 Extreme Cases. One extreme scenario is when the food is surrounded by walls while the GOAP agent has the Energy restriction. This makes surviving much easier for the RL agent on higher wall density maps since there is a greater chance the GOAP agent will become exhausted and be unable to refill its energy. In contrast, a smaller observation window for the RL agents worsens its ability to avoid the GOAP greatly since it has less time to react to its presence and craft a policy to effectively deal with the pursuing agent.

6 Discussion

6.1 Scalability of Tabular Q-Learning

The most significant limitation of the RL agent in this simulation is the scalability of tabular Q-learning. The Q-table grows with every new state-action pair the agent encounters, and the total theoretical state space is already 3,375 entries for the default 9×9 window. Increasing the window size to 13×13 pushes this to 7,225 entries for the position component alone, not accounting for the additional bucketed dimensions. In practice, many of these states are never visited on a fixed 16-by-16 map, so the table stays smaller than the theoretical ceiling, but the problem becomes more apparent in a randomized environment where new wall layouts continuously expose the agent to states it has not encountered before. The deeper issue is that the Q-table has no way to generalize between similar but distinct states. Two positions that are one cell apart are treated as entirely independent entries, so the agent has to visit each one individually before it can develop a policy for it. This is a known limitation of tabular methods and is the primary reason deep reinforcement learning approaches, such as Deep Q-Networks, replace the table with a neural network that can generalize across nearby states [2].

6.2 The Effect of GOAP Modules on RL Convergence

Initially, the GOAP was dominant throughout the simulation because the RL agent has a simple reward function based on its Manhattan distance from the pursuer. The reimplementation of the reward function with the desire to have kiting behavior emerge was one of the most impactful decisions on this project. The change in feedback helped the RL agent understand and present kiting behavior more consistently, and it became a more effective evader because of it.

The results from the five scenarios reveal a consistent pattern: the more modules the GOAP has active, the harder it is for the RL agent to converge on a reliable policy during training. In Scenario A, with no modules, the GOAP is entirely deterministic, and the RL agent can develop a movement loop that reliably exploits it. Adding rest and energy, as in Scenarios C and D, introduces nondeterminism that breaks the predictability the RL agent depends on to lock in a loop. The pounce action is particularly disruptive because it collapses the 3-cell safe distance at the edge of the zone reward band, which is precisely where the agent is incentivized to position itself. The RL agent does not have the capacity within a 9-by-9 window to reliably distinguish when a pounce is coming and pre-emptively increase its distance, which is why Scenarios C and E see the lowest training survival rates.

This also highlights a mismatch between the reward function's zone band and the GOAP's pounce range. The lower bound of the zone $d_{\min} = 3$ places the agent at the exact distance a pounce can reach. A practical adjustment would be to raise $d_{\min}$ to 4 when the pounce module is active, pushing the agent just outside pounce range while still keeping it close enough to control the GOAP's movement. This kind of module-aware reward shaping would be a natural direction to explore in future iterations of the simulation.

6.3 Fixed vs. Randomized Environment

The difference in agent performance between fixed and randomized environments is primarily a question of whether the RL agent is memorizing or generalizing. In a fixed environment, the agent can learn very specific movement patterns tied to the wall layout of that particular map, which is why Scenario A produces high survival rates in testing even though training survival is low. The agent is essentially memorizing a loop that works on that map. In a randomized environment, no such memorization is possible, and the agent is forced to develop strategies that transfer across layouts. Scenario D shows that this is achievable without the pounce module, with the agent learning to use the open border cells and maintain spacing. However, Scenario E shows that adding pounce back in makes generalization across random maps much harder, since the agent can no longer rely on a consistent safe distance.

6.4 Partial Observability

Another limitation worth noting is that the RL agent's observation window only captures the relative position of the GOAP, not the wall structure of the map around it. This means the agent cannot reason about whether it is approaching a dead end or an open corridor, and its mobility count is the only proxy it has for local topology. The mobility penalty in the reward function partially compensates for this by discouraging positions with few passable neighbors, but it does not give the agent any look-ahead capability. A larger window or an explicit representation of nearby wall cells in the state would give the agent more information to act on, though it would also expand the state space substantially.

In turn, the RL agent could be developed as a model-based algorithm. Since the model is being trained in a simulator, the agent could theoretically be given the map of the environment and the GOAP agent's path while being trained or when being tested One could argue that this would defeat the purpose of observing whether or not an RL agent can present the behavior of kiting from a limited information given to it as it would feel less like a player of a game attempting to avoid an enemy within that game. In contrast, another could argue that players are aware of how games are programmed and, by extension, how the enemies in the game move and behave. In either case, a model-based extension of this project would be a direction research in this area could be taken.

7 Conclusion

This paper described the design and implementation of a pursuit-evasion simulation in which a Goal-Oriented Action Planner acts as a pursuer and a tabular Q-learning agent learns to evade it. The GOAP was given three optional modules, pounce, rest, and energy, that can be independently toggled to change how difficult and unpredictable it is as a pursuer. The RL agent's reward function was shaped to encourage kiting behavior specifically by rewarding the agent for staying in a distance band of 3 to 6 cells from the GOAP and penalizing it for entering corners or narrow corridors.

The results across five scenarios confirm that kiting does emerge as a learned strategy. In the simplest cases the agent reliably exploits the GOAP's deterministic movement by developing oscillation loops that keep it in the reward zone indefinitely. As more modules are added the GOAP becomes harder to predict, and the RL agent's training survival drops accordingly. The pounce module proves to be the single most disruptive addition since it directly threatens the distance band the reward function is designed to maintain.

The main limitation of the approach is the scalability of tabular Q-learning. The table cannot generalize between nearby states, which means the agent requires many episodes to cover enough of the state space to act well, and its performance degrades on maps it has never seen before. Replacing

the Q-table with a neural function approximator, or enriching the state representation to include local wall structure, would be the most impactful next steps. A module-aware reward function that adjusts the safe distance band based on which GOAP actions are active would also likely improve convergence, especially in scenarios where the pounce module is enabled. Outside of improvements within this simulation, full implementation of a model-based learning agent could provide different and interesting insights that could be explored.

References

[1] Mat Buckland. 2014. *Programming game AI by example.* Wordware Publishing, Inc.

[2] GeeksforGeeks. 2025. https://www.geeksforgeeks.org/machine-learning/q-learning-in-python/

[3] Rufus Isaacs. 2012. *Differential Games: A mathematical theory with applications to warfare and pursuit, control and optimization.* Dover Publications.

[4] Jeff Orkin. 2006. Three states and a plan: The A.I. of F.E.A.R. – game developers conference 2006. https://www.gamedevs.org/uploads/three-states-plan-ai-of-fear.pdf

A Code and Dataset Links

Link: Google Drive Folder

The link above leads to a Google Drive folder containing the file used to run the project.